Listing 10.5 Using the `AuthorizePage` method to apply authorization to a specific page

```
builder.Services.AddRazorPages(options => {
    options.Conventions.AuthorizePage("/Index");                    ❶
    options.Conventions.Add(new CultureTemplatePageRouteModelConvention());
    options.Conventions.Add(new PageRouteTransformerConvention(new
KebabPageRouteParameterTransformer()));
});
```

❶ Using AuthorizePage with no policy has the same effect as applying a plain Authorize attribute to the endpoint PageModel.

You only want authorized users to be able to reach your CRUD admin pages in the various *Manager folders in the Pages directory. You can use the `AuthorizeFolder` method to immediately bar unauthenticated users from access, as in the following listing.

Listing 10.6 Authorizing folders with the `AuthorizeFolder` method

```
builder.Services.AddRazorPages(options => {
    options.Conventions.AuthorizeFolder("/CityManager");
    options.Conventions.AuthorizeFolder("/CountryManager");
    options.Conventions.AuthorizeFolder("/PropertyManager");
});
```

10.1.2 Allowing anonymous access

There may well be occasions when you want to bar anonymous access to most of an application's content but enable it for the odd page. For example, you might be developing an internal line-of-business application that needs to be locked down to prevent anonymous users from gaining access. You can do this by specifying a `FallbackPolicy` that requires authentication as part of configuring authorization services:

```
builder.Services.AddAuthorization(options => {
    options.FallbackPolicy = new AuthorizationPolicyBuilder().
```